Reporting Tables Implementation

Sprint 8: Newspapers, guest_searches, magazineReported, newspaperReported

Overview

This report explains data preservation methods used for the assigned tables.

Dumps were created in two categories based on table size, volatility, and retention rules.

```sql
SELECT
  table_name,
  table_rows
FROM information_schema.tables
WHERE table_schema = 'dc'
  AND table_name IN (
    'Newspapers',
    'guest_searches',
    'magazineReported',
    'newspaperReported'
  );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TABLE_NAME	TABLE_ROWS
Newspapers	37
guest_searches	0
magazineReported	0
newspaperReported	0

Table	Size	Dump Type	Notes
Newspapers	Small	Full Dump (A)	Includes structure + all data and manipulations

guest_searches	EMPTY	NONE	-
newspaperReported	EMPTY	NONE	-
magazineReported	EMPTY	NONE	-

Retention check: All tables were checked for records older than 90 days. Any violations were flagged and will be escalated to the backend teams and sponsors.

Dump Details

<u>Dump A</u> → Small Tables

- Tables included: Newspapers, guest_searches, magazineReported, newspaperReported
- Export type: Structure + Data
- File names:
    - dc_structure-dump_2026-02-28_Newspapers.sql
    - dc_data-dump_2026-02-28_Newspapers.sql
- Purpose: Provides full data for rollback, testing, and validation for smaller, stable tables.

<u>Dump B</u> → LargeTables (NONE)

- Tables included: NONE
- Export type: Structure only (no data)
- File name: None
- Purpose: Preserves table schema and indexes for large, volatile scraping tables without storing millions of rows of temporary data.

Notes:
Triggers, events, and stored procedures were excluded from the schema-only dumps because they are not required for rollback or performance testing.
Refer to sprint 4 reporting optimization documentation for a detailed explanation of table schemas, stored procedures and events.

guest_searches, newspaperReported, and magazineReported (Empty Tables)

guest_searches

```
1 •  SELECT * FROM dc.guest_searches;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ip_address	search_count	last_search_time
NULL	NULL	NULL

newspaperReported

```
1 •  SELECT * FROM dc.newspaperReported;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

idnewspaperReported	idEmailAlert	newspaperID
NULL	NULL	NULL

magazineReported

```
1 •  SELECT * FROM dc.magazineReported;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

idmagazineReported	idEmailAlert	magazineID
NULL	NULL	NULL

- These empty tables can be deleted. They have no foreign keys and no relationships with other tables as well

Download / Excluded Data (NONE)

Some data was excluded from dumps due to size and retention rules.

These are the steps followed to identify such tables:

1. Identified data that cannot be included in SQL dumps
    a. Historical scrape data from over 90 days
    b. Large result sets that would create huge files
2. Downloaded and exported data separately”
    a. Format: csv file
3. Stored securely in:
    a. /dumps folder in Github repository
    b. Explicitly named: NONE
    c. Ensured it is accessible for future validation

Notes:
The empty tables need to be deleted. The Newspapers table consists of a list of newspapers and doesn’t have any foreign keys.

Trigger Checks

None of my assigned tables have triggers.

SRT Validation

None of the tables assigned to me this sprint are SRT tables.

Next Steps

- Verify that dumps restore correctly in test environments.
- Document any retention violations and escalations
- Track dumps and documentation from other teams
- Make optimization changes to the dumped tables

Summary

- Large, volatile tables (Results) → schema-only dumps
- Small, stable tables (Feeds) → full dumps
- Empty tables → no dumps, can be deleted
- Excluded/large data → downloaded separately, stored securely

- Documentation ensures traceability, rollback support, and compliance with retention rules
- No triggers in the assigned tables